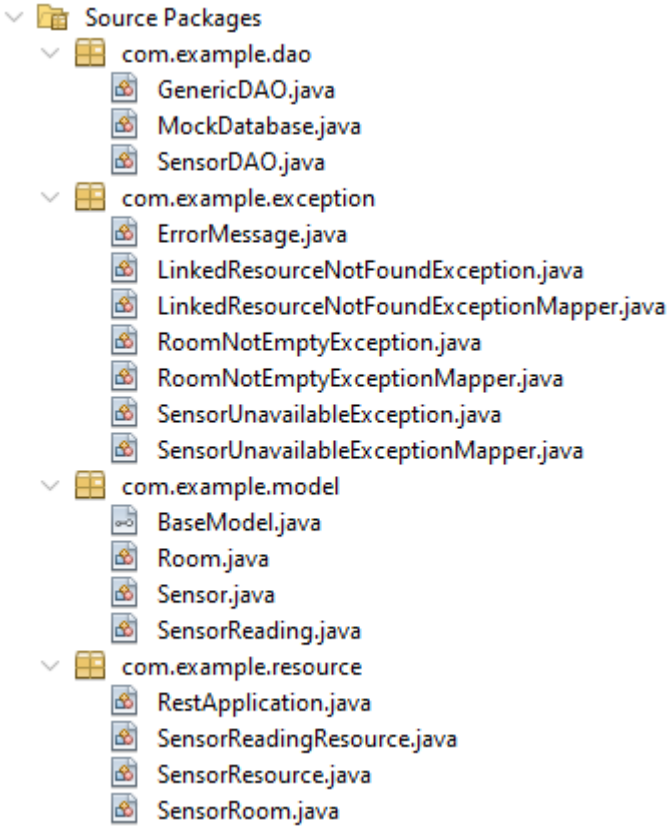
Client Server Architectures
Muhammed Iyaas Choudhury
W19678180

Overview

My API design essentially follows the tasks of the coursework closely. It holds the three classes given to us and separates each distinct java class in corresponding java packages



as shown in the image:

The API has full functionality however it does not complete all tasks of the coursework as I did not fully complete part 4 and part 5 of the coursework more specifically task 2 for part 4 and tasks 4 and 5 of part 5 were not completed.

In terms of functionality the API allows users to view, add and delete rooms as well as view sensors and specifically search for a sensor similarly to specifically searching for a room based on the id of that room. As well as allowing users to check sensor reading whilst also specifically checking individual sensors.

All functionality comes with each task allocated excluding the tasks that were stated to not have been completed earlier.

To run the project all source packages must be organised as seen in the image above with the Apache Tomcat or TomEE service running.

Sample curl commands

POST http://localhost:8080/ClientServerCW/api/v1/rooms Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
    "id": "SWI-304",
    "name": "Swimming Pool",
    "capacity": 30
}
```

Body Cookies Headers 6 Test Results 0/1 201 Created 74 ms 283 B Save Response

{} JSON Preview Visualize

```
{
    "id": "SWI-304",
    "name": "Swimming Pool",
    "capacity": 30,
    "sensorIds": []
}
```

Above presents a post command adding a room.

DELETE http://localhost:8080/ClientServerCW/api/v1/rooms/SWI-304 Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
{
    "id": "SWI-304",
    "name": "Swimming Pool",
    "capacity": 30
}
```

Body 204 No Content 10 ms 112 B Save Response

Raw Preview Visualize

```
1
```

Above presents a delete command deleting a room that has no active sensors.

DELETE http://localhost:8080/ClientServerCW/api/v1/rooms/LIB-301 Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{

    "id": "SWI-304",
    "name": "Swimming Pool",
    "capacity": 30



}
```

Body Cookies Headers 5 Test Results 0/1 409 Conflict 14 ms 295 B Save Response

{} JSON Preview Debug with AI

```
{
    "message": "Room is currently occupied with active sensors",
    "status": 409,
    "documentation": "https://myuniversity.edu/api/docs/errors"
}
```

Above presents a delete command attempt and a 409 response when attempting to delete a room with active sensors.

GET http://localhost:8080/ClientServerCW/api/v1/sensors?type=Occupancy Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

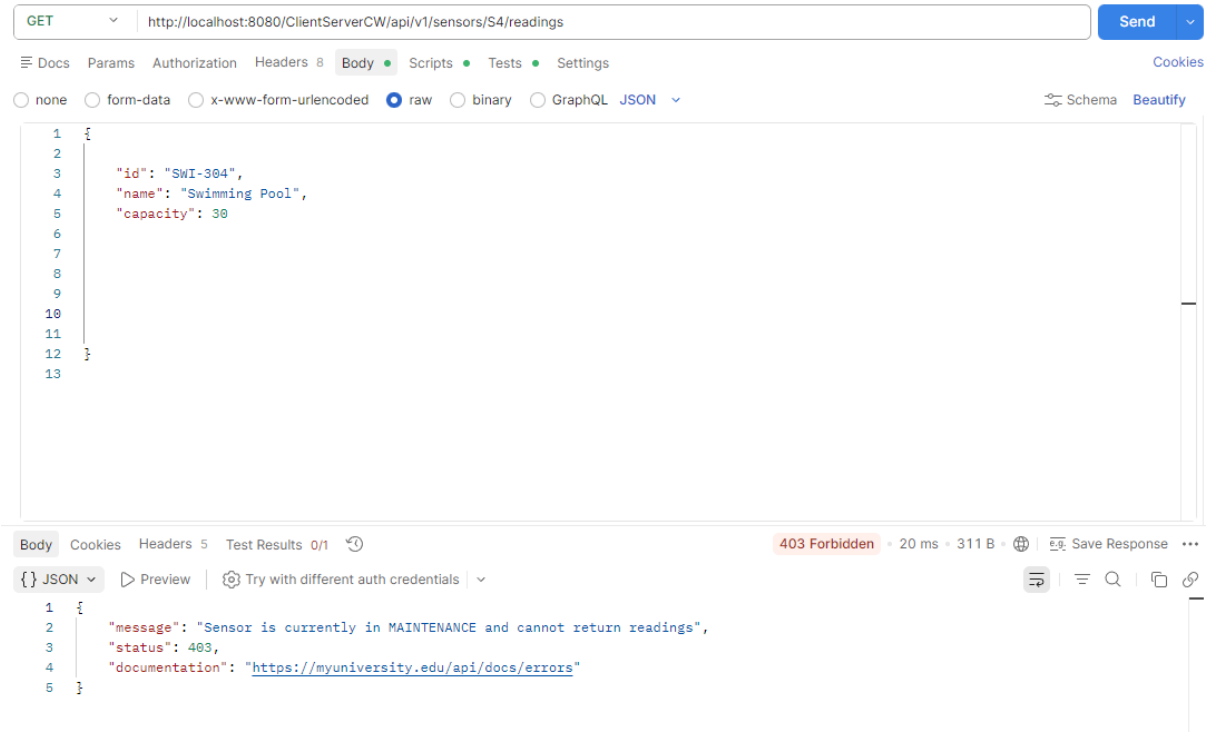
```
{

    "id": "SWI-304",
    "name": "Swimming Pool",
    "capacity": 30



}
```

Body Cookies Headers 5 Test Results 1/1 200 OK 5 ms 339 B Save Response

{} JSON Preview Visualize

```
[
    {
        "id": "S2",
        "type": "Occupancy",
        "status": "ACTIVE",
        "currentValue": 30.0,
        "roomId": "REC-302"
    },
    {
        "id": "S4",
        "type": "Occupancy",
        "status": "MAINTENANCE",
        "currentValue": 30.0,
        "roomId": "ASS-303"
    }
]
```

Above presents a get command gathering the type specific sensors available.



Above presents a sensor id specific get request and also a 403 error response.

Questions:

Part 1

1.
-Question: In your report, explain the default lifecycle of a JAX-RS Resource class. Is a new instance instantiated for every incoming request, or does the runtime treat it as a singleton? Elaborate on how this architectural decision impacts the way you manage and synchronize your in-memory data structures (maps/lists) to prevent data loss or race conditions.

A resource class' lifecycle is determined through the user requests. This means that a resource class is made when requested by the user.

The runtime does not reuse the same object for multiple requests. This means that once the request is processed the instance is then discarded.

The default lifecycle has a significant effect on data structures like maps and lists that are used. Any instance variables utilised are recreated with each request. This means that the data stored is temporary as it will be lost after the requests.

2.
Why is the provision of "Hypermedia" (links and navigation within responses) considered a hallmark of advanced RESTful design (HATEOAS)? How does this approach benefit client developers compared to static documentation?

HATEOAS is considered more advanced because the API describes itself.

Rather than relying on external data the API would include links that guide the client.
This makes the client development easier as well as the overall reduction of errors.

Part 2

1.
When returning a list of rooms, what are the implications of returning only IDs versus returning the full room objects? Consider network bandwidth and client side processing.

Returning only the room IDs will hold no benefit to the client as it holds no information other than the amount of rooms when all the other attributes are still logged with each room.

2.
Is the DELETE operation idempotent in your implementation? Provide a detailed justification by describing what happens if a client mistakenly sends the exact same DELETE request for a room multiple times.

If a client attempts to delete request multiple times it will work initially if there are no active sensors, however if attempted in the same session once the room has already been deleted they will receive a 500 internal server error due to the room already being deleted.

Part 3

1.
We explicitly use the @Consumes (MediaType.APPLICATION_JSON) annotation on the POST method. Explain the technical consequences if a client attempts to send data in a different format, such as text/plain or application/xml. How does JAX-RS handle this mismatch?

It will not go through and an error will appear as information can only be saved specific to how data is already logged.

2.
You implemented this filtering using @QueryParam. Contrast this with an alternative design where the type is part of the URL path (e.g., /api/vl/sensors/type/CO2). Why is the query parameter approach generally considered superior for filtering and searching collections?

When specifically filtering through attributes in this case the category it makes sense to query the specific amount on the specific collection rather than loading the attribute as a sub-resource in the url when in the database it is only an attribute within rooms.

Part 4.

1.
Discuss the architectural benefits of the Sub-Resource Locator pattern. How does delegating logic to separate classes help manage complexity in large APIs compared to defining every nested path (e.g., sensors/{id}/readings/{rid}) in one massive controller class?

The Sub-Resource Locator pattern is essentially a way to break a big API into smaller, more focused pieces instead of stuffing everything into one huge controller.

Without this one large class is formed handling all nested routes making it messy and making it hold too many responsibilities.

With Sub-Resource Locator each attribute handles its own data making everything simpler and easier to read.

Part 5

2.
Why is HTTP 422 often considered more semantically accurate than a standard 404 when the issue is a missing reference inside a valid JSON payload?

HTTP 422 is more specific to the cause of the error. 404 is known for the entirety of the data searched for not being found whereas 422 is indicating a specific piece of data is missing which in this case within the post request once it is made clear that the roomId is required once a new sensor is attempted to be added going forward there will be no further errors relating to lost or not found data.